Jeda Krisnell Dionisio
COGS 181

Convolutional Neural Networks for Fake Audio Detection

Abstract

AI-generated audio is becoming incredibly common, and there is a need for reliable detection methods. This study investigates the use of convolutional neural networks (CNNs) for classifying real and fake audio. There is a concentration on the tweaks that can be make in architectural and hyperparameters for a greater model. Using a subject of the SceneFake dataset (5000 samples), 10 CNN models were developed. These had varying activation functions, pooling strategies, optimizers, and regularization techniques. Results suggest model performance varying significantly. The best configuration of this project achieved a 79.6% accuracy. A few achieved accuracy that were similar to near random chance at 50%. The following consistently improved performance: ReLu, Adam, and dropout. Learning rate and weight decay influenced the tradeoff between model stability and accuracy. CNNs can effectively detect manipulated audio.

I. Introduction

Fake audio detection represents a foundational task for the present and future as generative artificial intelligence and the like have become increasingly better at posing as a human. Internet users have interacted with AI-generated images, listened to AI-generated music, and watched deepfakes of famous celebrities. Some content can even include both “real” and “fake” components alike. Neural networks are often related to the creation of “fake” content. This artificial mathematical model is meant to mimic the human brain by processing information and learning patterns to accomplish tasks. Not only can neural networks create new content, but the model can also help with detection.

The central research question is the following: How can convolutional neural networks (CNNs) be used to detect fake and real audio files? To gain a greater and more complete understanding of using CNN, the question of which activation functions, pooling functions, and optimization choices are also posed. Which are going to lead to a more accurate model? What are the tradeoffs? In hopes to answer these questions and more, I conduct a comprehensive exploration of the SceneFake dataset and various combinations of activation functions, regularization techniques, and optimization choices.

II. Methods

A. Dataset

The first step to solving this problem is getting data. The dataset that I used to complete this project is from “SceneFake: An Initial Dataset and Benchmarks for Scene Fake Audio Detection” by Yi et al. (2024). The authors introduce a dataset containing nearly 6GB of audio files (WAV) in three separate categories (train, development, and evaluation). Within each category, there are real and fake audio files respectively. An audio file is usually about 5 seconds long with a speaker saying 1-2 sentences with some noise in the background. A clip is labeled “fake” when the audio in the background is being manipulated as opposed to the manipulation of the speaker’s voice. To reiterate, the speaker’s voice is completely real, and the noise in the background is what will be altered.

Because of the vastness of the SceneFake dataset and limitations from my computing power, I had to limit the nearly 66,000 audio files into a more manageable size. My final dataset composed of 5000 WAV files. There are 3750 training files and 1250 testing files for a 75/25 test/train split. To break this down even further, there are 1875 fake and 1875 real clips for training. The same 50/50 split can be found for testing. I decided not to use the development category as I am not fine tuning the models based on batching or learning rate. I will prioritize other means of hyperparameters. The files from the original dataset were randomly selected to be included in the working dataset for this project.

B. Audio Convolutional Neural Network Models

As part of my research towards utilizing CNN for fake audio detection, I experimented on various audio neural network architectures that varied in hyperparameters. The architecture was motivated by the published blog post by codegenes in 2025 for the first two models (Model 1 & 2). The rest were up to my discretion. I continued to learn about audio CNN reading Nandi (2021). The hyperparameters that were to be experimented on were the following: activation functions (sigmoid, ReLU, leaky ReLU), pooling functions (average, max, adaptive average), optimization (stochastic gradient descent (SGD), Adam), regularization (dropout), and normalization (batch normalization). Each model had the same training process with 10 epochs. They also had the same loss function with cross entropy loss. The average mini-batch loss was recorded for each model for a visual. The validation loss was taken as it did not occur to me that I could do that until it was too late. I had no time to rerun the models as some of them spent more than 2 hours to train. Regardless, below are the descriptions of each model that was trained. The model was tested with new data it had not seen before after training. The accuracy percentage was calculated.

Model 1: This neural network has a total of **5 layers**. There is 1 `Conv2d`, 1 `MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was not** used. **Batch normalization** was not used. This neural network is motivated by the blog post mentioned earlier (codegenes 2025). I started learning about audio neural networks through this article for the purpose of this project. I aspired to use this as the foundation going forward, gaining more knowledge through trial and error.

Model 2: This neural network has a total of **5 layers**. There is 1 `Conv2d`, 1 `MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was** used. Batch **normalization was not** used. This neural network is similarly motivated by codegenes (2025). This eventually become my blueprint for a few of the models.

Model 3: This neural network has a total of **6 layers**. There is 2 `Conv2d`, 1 `AvgPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **Sigmoid**. The pooling functions were **average** and **adaptive average**. The optimizer that was used **SGD**. Dropout **regularization was not** used. Batch **normalization was not** used.

Model 4: This neural network has a total of **5 layers**. There is 1 `Conv2d`, 1`MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **SGD**. Dropout **regularization was not** used. Batch **normalization was not** used.

Model 5: This neural network has a total of **5 layers**. There is 1 `Conv2d`, 1 `MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **leaky ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was** used. Batch **normalization was not** used.

Model 6: This neural network has a total of **5 layers**. There is 1 `Conv2d`, 1 `AvgPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **average** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was** used. Batch **normalization was not** used.

Model 7: This neural network has a total of **10 layers**. There is 4 `Conv2d`, 3 `MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was** used. Batch **normalization was** used. This model was vibe-coded as an experiment to see the accuracy between a human and large-language model.

Model 8: This neural network has a total of **6 layers**. There is 2 `Conv2d`, 1 `MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was** used. Batch **normalization was not** used.

Model 9: This neural network has a total of **7 layers**. There is 3 `Conv2d`, 1 `MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was** used. Batch **normalization was not** used.

Model 10: This neural network has a total of 6 **layers**. There is 2 `Conv2d`, 1 `MaxPool2d`, 1 `AdaptiveAvgPool2d`, and 2 fully connected linear layers. The activation function of choice was **ReLU**. The pooling functions were **max** and **adaptive average**. The optimizer that was used **Adam**. Dropout **regularization was** used. Batch **normalization was not** used. The change for this neural network is in the learning rate and weight decay. They were significantly smaller (the others used a learning rate of 0.001 and no weight decay). This neural network had a **learning rate of 3e-4** and a weight decay of **1e-4**.

III. Experiment Results

The accuracies varied across the models, and one remained greater than the others. After patiently waiting for all the models to run through the 10 epochs, the accuracies were taken as well as a graph of the average mini-batch loss during the training process. Below are the accuracies. The visualizations are below References in the Figures section. To highlight the results of the models, I will discuss Models 1 & 2, Models 3-7, Models 8 & 10, and Model 9 separately.

Model 1: 55.20%
Model 2: 56.32%
Model 3: 50%
Model 4: 48.64%
Model 5: 50%
Model 6: 50.80%
Model 7: 50%
Model 8: 68.88%
Model 9: 79.60%
Model 10: 68.80%

A. Models 1 & 2

These two models have been grouped together because of their similarity. Both of the models had around the same accuracy (in comparison to the other model results), and their average mini-bass loss graph looks relatively the same. They both decrease over time, which shows learning and improvement. However, there are still ways to differentiate the two. The second model has dropout

regularization. Because of the inclusion of this hyper-parameter, the model increase by 1.12%. While this is not a huge change, the result displays how one hyper-parameter can influence training a model. The second graph also is noisier than the spikes going up and down. In the context of fake sound detection, including dropout has a better accuracy of identifying fake audio to real content than excluding it in the model.

B. Models 3-7

These 5 models have been grouped together because of their similar accuracy scores. The 5 of them had accuracies that were around 50% with the worst being Model 4 at 48.64%. This suggests that the model, and the others, are not really learning at all. This can be seen through the loss curve where there are no clear downward trends. The models are better off flipping a coin and deciding. In fact, Model 4 is worse at predicting the label of “fake” or “real” for audio files than a fair coin. While looking at the training curve, there is a (somewhat) downward trend. However, the line shows many staggered up and downs. The model may be improving in probabilities, but it is not making any better decisions. Something similar can be said about Model 6. Compared to the others, there is a more obvious downward trend in the training curve. There is still a lot of noise. Yet, this model is technical best out of the grouped 5 with a 0.80% increase.

By looking into the neural network hyperparameters, the interactions between the number of layers, activation functions, pooling functions, optimization, regularization, and normalization can be investigated. The models will be examined by their number of layers. Models 4-6 had 5 layers each. Although there were meaningful differences between the three models (leaky ReLU vs. ReLU, Adam vs. SGD, max pooling vs. average pooling, dropout regulation or not), the three are still ~50%. These models exhibit that the architecture almost had no impact. There is some improvement with Adam and dropout together, though. From this, I learned that Adam and dropout are generally good for working with this audio detection task. Model 3 had 6 layers. This model used both Sigmoid and SGD. As mentioned in class, Sigmoid is not the greatest activation function. This is from the vanishing gradients and slow learning. The loss graph and detection accuracy show this. I learned that Sigmoid should not be used for this audio detection task.

Finally, Model 7 has 10 layers. I was disappointed in this model as it was vibe-coded, and I would have expected a better accuracy because of it. Nevertheless, this was the first model to introduce batch normalization. While batch normalization is generally effective in convolutional neural networks, in this model, there was no difference than random classification. The training and test sets may have negatively impacted this architecture. The code implemented for the datasets were consistent throughout the entire project. I am unsure why this model had this accuracy. This experiment was a gentle reminder that vibe-coding may have faults.

C. Models 8 & 10

These 2 models have been grouped together because of their similar accuracy scores. They both essentially the same model; however, the learning rate and weight decay are different. The accuracy scores were within 1% of each other, but looking at the graphs, Model 10 looks a lot more “stable” than Model 8. In the former, there is an obvious downward trend that has a dramatic spike near the end of training. This suggests the model is more stable and calibrated, but it is not as good as detecting fake files. In the latter, there is still downward trend, but there are many more dramatic spikes. This suggests the model is better overall, but makes a few very confident mistakes in detecting fake audio files. This would inflate the loss.

Seeing the differences between the two models exhibited the importance of learning rate and weight decay. For Model 10 with the slower learning and weight decay, there is stability. That means that the updates are smaller and the loss is smoother. At the same time, the model is less exploratory. There are fewer spikes and worse final accuracy, which is exactly what happened. When it comes to

weight decay, there is a greater generation, but it can cap on the performance. Weight decay also prevents overfitting, which is necessary in my case since my dataset is so small relative to the original size. Regardless, weight decay results in less overconfident predictions for audio detection. Because of these two models, I was able to see firsthand the influences of hyperparameters, explicitly learning rate and weight decay. In future models, I plan on utilizing this for greater accuracy.

D. Model 9

With the addition of all the information I've gathered from other models, I created a model and included the hyperparameters that I believed would lead to the greatest accuracy. This said model has the highest accuracy. While this may be the case, the loss graph is also one of the more interesting ones. There is a downward trend from the beginning of start of the training to the end. Yet, there are many spikes in the graph itself. This suggests that the model has a better time identifying a fake audio file, but it occasionally encounters batches that it handles very poorly. Looking at the architecture, there are 7 layers. Namely, there are 3 `Conv2d` layers that may have allowed it to improve in accuracy overall while adding the optimization and regularization. I recognize that there are ways to improve the accuracy, and I will use this model as a blueprint going forward.

IV. Limitations and Future Directions

There are limitations to consider regarding this project. It is important to recognize the sample size relative to the original dataset. There is a great computational loss because of the decision to cut down to number of total files used. The smaller size of the dataset can also lead to overfitting the model. Finally, 10 epochs were used for refining and learning. This may not have been enough for the context of this project. As a solution to the limitations, I offer future directions for this project. This includes running the model on the entire original dataset. There would be more files to train, validate, and test on. More on this point, it would be crucial to validate the model. I would also suggest more processing power with GPUs and other resources to run the model. With this increase in power, more epochs would be able to run. All of this would potentially lead to a greater model for detecting fake audio. I plan on continuing to improve this model and experience in audio convolutional neural networks to detect generative AI music.

V. Conclusion

The experiments that were conducted in this project demonstrate convolutional neural networks can effectively detect fake and real audio files. The performances of 10 models exhibited the strengths and weaknesses of utilizing several different hyperparameters and architectural choices. Model 9 achieved the highest accuracy at 79.6%, suggesting that 3 `Con2d` layers with the ReLU activation function, max pooling and adaptive max pooling, Adam optimization, and dropout regularization are a great balance for this task. This conclusion can be made in comparison to the 9 other models.

By analyzing the loss curves, there are tradeoffs between accuracy and stability. The one model with the lower learning rate and weight decay had the smoothest and most stable loss curve. The rest of the models produced spiky loss curves with overconfident misclassifications. These also highlight the importance of hyper-parameter tuning. There is a meaningful impact. Sigmoid activations and SGD were performing with a near random chance.

These results demonstrate that even shallow CNN architectures can achieve strong performance in detecting manipulated/fake audio. It is promising as the growth of generative AI applications can manipulate, misinform, and trick online users. There is an increasing need for reliable detection methods. In the context of audio detection, convolutional neural networks can work.

References

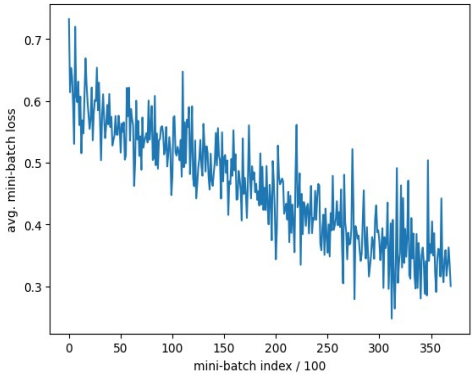
Audio convolutional neural networks with pytorch. (2025, November 13). codegenes. https://www.codegenes.net/blog/audio-convolutional-neural-network-pytorch/.

Nandi, P. (2021, March 24). Cnns for audio classification. Towards Data Science. https://towardsdatascience.com/cnns-for-audio-classification-6244954665ab/.

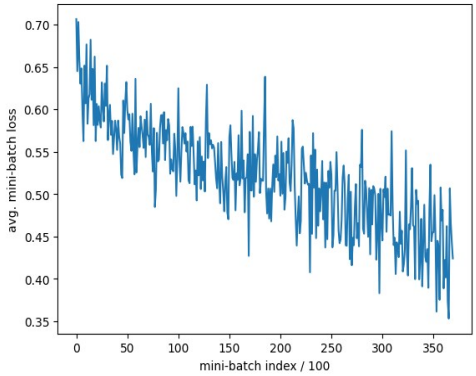
PyTorch documentation — PyTorch 2.10 documentation. (n.d.). https://docs.pytorch.org/docs/stable/index.html.

Yi, J., Wang, C., Tao, J., Zhang, C. Y., Fan, C., Tian, Z., et al. (2024, April 4). SceneFake: An Initial Dataset and Benchmarks for Scene Fake Audio Detection. arXiv. https://doi.org/10.48550/arXiv.2211.06073

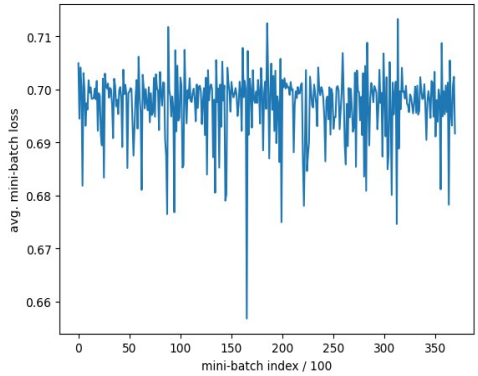
Figures



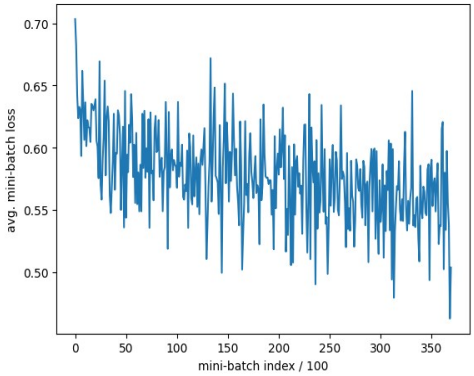
Model 1



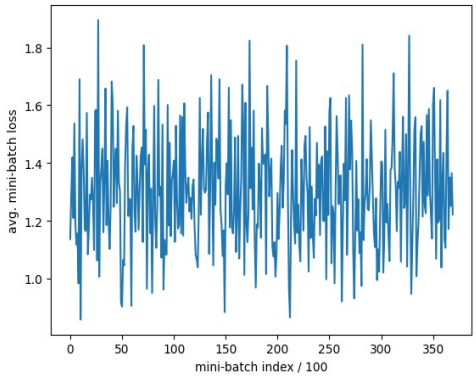
Model 2



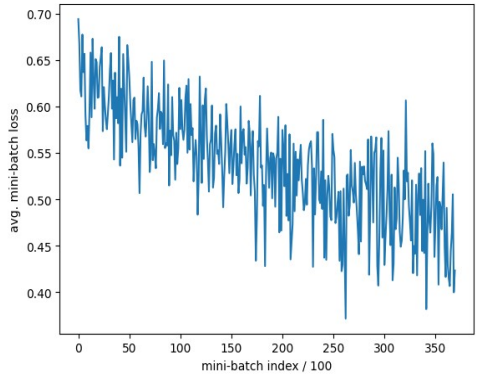
Model 3



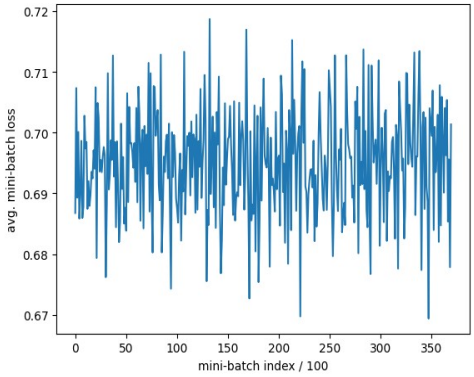
Model 4



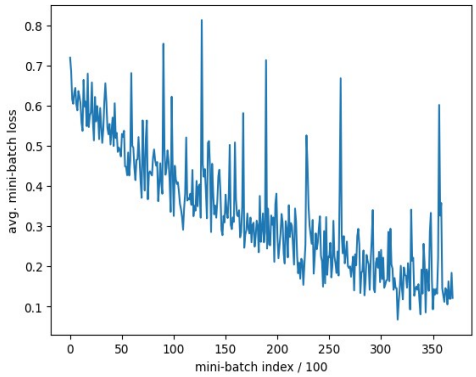
Model 5



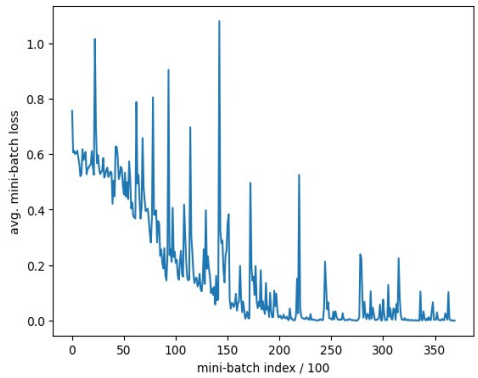
Model 6



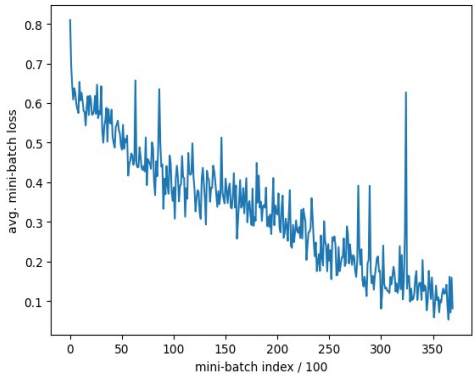
Model 7



Model 8



Model 9



Model 10